

KYC Research Agent - Tech Stack & Process Flow

Overview

A Streamlit-based KYC (Know Your Customer) research application that automatically gathers comprehensive company information from multiple sources including Wikipedia and Google search results with intelligent caching capabilities.

Tech Stack

Core Framework

- **Streamlit**: Web application framework for the user interface
- **Python 3.x**: Primary programming language

Web Scraping & Data Collection

- **Requests**: HTTP library for API calls and web scraping
- **BeautifulSoup4**: HTML parsing and content extraction
- **Serper API**: Google search integration (requires API key)
- **Wikipedia API**: Direct API access for comprehensive Wikipedia content

Data Processing & Storage

- **Pandas**: Data manipulation and analysis
- **JSON**: Metadata storage and caching
- **Pathlib**: File system operations
- **Regular Expressions (re)**: Text validation and sanitization

External Dependencies

- **data_scraper.get_data_ch**: Custom module for company house data search
- **Environment Variables**: API key management (SERPER_API_KEY, OUTPUT_DIR)

UI/UX Enhancement

- **Custom CSS**: Material Design styling (styles.css)
 - **Streamlit Components**: Interactive widgets, progress bars, metrics
-

Process Flow

Phase 1: Company Discovery

User Input → Validation → Company House Search → Results Display

1. **Input Validation:** Checks for meaningful text (>2 chars, contains letters)
2. **Company Search:** Uses `get_data_ch.search_all()` to find matching companies
3. **Results Display:** Shows filtered results in required columns (`title`, `company_number`, `company_status`, `address.country`)

Phase 2: Research Execution

Company Selection → Cache Check → Multi-Source Research → Data Aggregation

2.1 Cache Management

- **Cache Check:** Validates existing research data (24-hour expiry)
- **Cache Structure:**
 - `{query}_metadata.json`: Metadata and timestamps
 - `{query}.txt`: Complete research content
- **Cache Decision:** Use cached data or perform fresh research

2.2 Multi-Source Data Collection

```
Wikipedia Scraping ← → Google Search (Serper API)
    ↓                      ↓
Complete Content    → Website Scraping
    ↓                      ↓
Data Aggregation & Storage
```

Wikipedia Integration:

- Uses multiple API endpoints for comprehensive data
- Extracts: Full content, categories, sections, metadata
- Handles content limits (up to 100k characters)

Google Search Integration:

- Serper API for search results (9 results default)

- Extracts: Titles, URLs, snippets
- Respects rate limits with delays

Website Scraping:

- Scrapes each search result URL
- Content cleaning (removes scripts, styles, navigation)
- Limits content to 5000 characters per site

Phase 3: Data Processing & Storage

Raw Data → Content Cleaning → Structured Storage → User Access

3.1 Data Structuring

- **Unified Format:** All sources compiled into single text file
- **Metadata Tracking:** Source URLs, timestamps, content statistics
- **Content Organization:** Wikipedia first, then Google results with scraped content

3.2 File Management

- **Output Directory:** Configurable via environment variable
- **Filename Sanitization:** Removes special characters, spaces
- **Content Structure:**

```
Header Information
=====
Wikipedia Section (Complete content)
=====
Google Results (with scraped content)
```

Phase 4: User Interface & Interaction

Results Display → Download Options → Cache Management

4.1 Progressive Disclosure

- **Step-by-step Interface:** Guided user flow
- **Expandable Sections:** Detailed source information
- **Progress Indicators:** Real-time scraping status

4.2 Data Access Options

- **Immediate Download:** Complete research file
 - **Cache Management:** View/delete existing cache files
 - **Detailed Metrics:** Source counts, content statistics
-

Key Features

Smart Caching System

- **Time-based Expiry:** 24-hour cache validity
- **Metadata Tracking:** Query, timestamp, source counts
- **Cache Validation:** Automatic freshness checks

Robust Error Handling

- **Input Validation:** Text quality checks
- **API Fallbacks:** Graceful handling of API failures
- **Content Limits:** Prevents oversized downloads

Flexible Input Methods

- **Dropdown Selection:** From search results
- **Manual Entry:** Any company name (uses first word for search)
- **Search Optimization:** Automatic keyword extraction

Comprehensive Data Collection

- **Wikipedia:** Complete page content with metadata
 - **Google Results:** Multiple sources with full content
 - **Content Statistics:** Character counts, section analysis
-

Configuration & Deployment

Environment Variables

- `SERPER_API_KEY`: Required for Google search functionality
- `OUTPUT_DIR`: Cache storage location (default: ./output)

File Structure

```
kyc_agent/
├─ main_app.py          # Main application file
├─ styles.css           # Material Design styling
├─ data_scraper/        # Custom company search module
├─ output/              # Cache storage directory
│   └─ *.txt            # Research content files
│   └─ *_metadata.json  # Cache metadata files
```

Dependencies

- External API dependency on Serper for Google search
 - Custom module dependency for company house data
 - File system write permissions for caching
-

Use Cases

- **Due Diligence:** Comprehensive company research
- **KYC Compliance:** Automated information gathering
- **Market Research:** Multi-source company intelligence
- **Background Checks:** Automated data collection and verification